Manejo de excepciones y validación de entradas (Bloque 7) / Incorpora excepciones y validación, y commitea tu avance [Actividad core]

✓ **Hecho:** Ver

📘 **UNIDAD U1.3** · Python aplicado para integración con IA

Incorpora excepciones y validación, y commitea tu avance

🏁 Esta es la última lección de la **Unidad U1.3**, y cierra un recorrido de tres bloques:

Bloque 5	Bloque 6	Bloque 7 · Hoy
		
Construiste una primera versión de tu módulo.	La refactorizaste en funciones claras y reutilizables.	Le incorporas manejo de excepciones y validación de entradas.

Con eso, tu **módulo queda completo.**

Hoy también das un paso más: hasta ahora tu módulo consumía un servicio de prueba, pensado para practicar el patrón sin depender de credenciales reales. Hoy lo apuntas al proveedor de modelo real que ya usaste desde **Postman** o **Thunder Client** en el **Bloque 4**, usando la credencial que protegiste en tu archivo **".env"**.

🛠️ Qué vas a incorporar hoy

🔎 Valida la entrada al principio de la función que arma la petición

Antes de armar la URL o el cuerpo de la petición, confirma que la pregunta que vas a enviarle al modelo no venga vacía; si viene vacía, no tiene sentido continuar.

📡 Envuelve el envío de la petición en un manejo de errores de conexión

Un "try/except" alrededor de "requests.post", con un mensaje claro sobre que el servicio no respondió.

📦 Envuelve la conversión de la respuesta en un manejo de errores de formato

Un "except" separado, específico para el caso en que "respuesta.json()" falle porque lo que llegó no es JSON válido.

🔐 Maneja el caso de un dato faltante en la respuesta ya procesada

Si el campo que buscas no está donde lo esperabas, captura ese error específico ("KeyError" o "IndexError") con un mensaje que diga exactamente qué faltó.

Puesto todo junto, tu módulo completo debería verse algo así:

```python
import os
import requests
from dotenv import load_dotenv

 load_dotenv()

 def armar_peticion(pregunta):
     if not pregunta:
        raise ValueError("La pregunta no puede estar vacía")
     api_key = os.getenv("GEMINI_API_KEY")
     modelo = "gemini-3.6-flash"
     url = f"https://generativelanguage.googleapis.com/v1beta/models/{modelo}:generateConte
     body = {"contents": [{"parts": [{"text": pregunta}]}]}
     return url, body

 def enviar_peticion(url, body):
     try:
        respuesta = requests.post(url, json=body)
     except requests.exceptions.ConnectionError:
        raise RuntimeError("No se pudo conectar con el servicio. Intenta de nuevo en unos s
     try:
        return respuesta.json()
     except requests.exceptions.JSONDecodeError:
        raise RuntimeError("La respuesta del servicio no llegó en el formato esperado.")

 def extraer_texto(datos):
     try:
        return datos["candidates"][0]["content"]["parts"][0]["text"]
     except (KeyError, IndexError):
        raise RuntimeError("La respuesta no trae el texto generado en el lugar esperado.")

 def obtener_dato_de_prueba(pregunta):
     url, body = armar_peticion(pregunta)
     datos = enviar_peticion(url, body)
     return extraer_texto(datos)
```

***Fíjate que cada función maneja solo el error que le corresponde a su propia responsabilidad**: la validación vive en **"armar_peticion"**, el error de conexión y de formato en **"enviar_peticion"**, y el dato faltante en **"extraer_texto"**. Eso no es casualidad: es la misma separación de responsabilidades del **Bloque 6**, aplicada ahora también al manejo de errores.*

Y quien llame a tu función principal necesita, a su vez, estar preparado para recibir cualquiera de esos

errores:

```
try:
     resultado = obtener_dato_de_prueba(pregunta)
     print(resultado)
 except (ValueError, RuntimeError) as error:
     print("No se pudo completar la operación:", error)
```

🧪 Cómo probar que tu manejo de errores realmente funciona

No basta con escribir el código y asumir que funciona: provoca cada error a propósito, uno a la vez, exactamente como se hizo en la demostración del **Bloque 5**.

Cambia momentáneamente tu credencial por una inválida y confirma que el mensaje que recibes es claro. Restaura la credencial correcta, y luego prueba con un texto de entrada vacío para confirmar que tu validación lo detiene antes de llegar a la petición. Al final, ejecuta tu módulo una vez más con una entrada válida, para confirmar que el camino exitoso sigue funcionando exactamente igual que antes.

Ese último paso, el de la entrada válida, es el que te dice si todo el camino sigue funcionando de principio a fin. En tu consola, el resultado exitoso se ve así (el texto exacto va a variar, porque viene de un modelo generativo real, pero no debería aparecer ningún traceback ni mensaje de error):

```
Sí, esta conexión funciona correctamente.
```



Esa captura debería mostrar tu terminal después de las tres pruebas seguidas: el mensaje de error de credencial inválida, el mensaje de validación por entrada vacía, y por último el resultado exitoso con una entrada válida, uno después del otro, sin que el programa se haya detenido sin control en ningún punto.

Qué hace que tu manejo de errores sea bueno, no solo presente

Ya viste esta idea en la primera lección de este bloque, y ahora es el momento de aplicarla: un **except** que solo evita que el programa se detenga, sin decir nada útil sobre qué pasó, no es un buen manejo de errores, es un problema escondido.

Revisa cada uno de tus mensajes de error: tienen que decir, con claridad, qué falló, no solo que "algo salió mal". Ese detalle es exactamente lo que la rúbrica de calidad de código evalúa en este criterio.

Posibles errores comunes: tu archivo ".env"

Si ves **RuntimeError: La respuesta no trae el texto generado en el lugar esperado** apenas ejecutas tu módulo, antes de sospechar de tu código revisa tu credencial. Con una API key ausente o inválida, el servicio no rechaza la conexión (por eso no ves el error de conexión), sino que responde igual, pero con un error de autenticación en vez de tu respuesta; el resultado es que **"extraer_texto"** no encuentra el campo **"candidates"** que esperaba, y ese es el mensaje que terminas viendo.

🔑 **Causa más común:** tu archivo **".env"** no está en la raíz de tu proyecto, no se llama exactamente **".env"**, o el nombre de la variable no es exactamente **GEMINI_API_KEY**.

✅ Verificación final de tu módulo completo

Antes de entregar, revisa tu módulo contra los cuatro criterios de la rúbrica que viste en la lección anterior, aplicados ahora a la versión completa:



Responsabilidad única

Cada función sigue haciendo una sola cosa, incluso después de agregarle el manejo de errores.



Nombres descriptivos

Los nombres siguen comunicando su propósito, incluyendo cualquier función o variable nueva que hayas agregado hoy.



Comportamiento correcto y verificado

El camino exitoso produce el mismo tipo de resultado que en tu prueba manual del **Bloque 4**, y ya lo confirmaste ejecutándolo.



Manejo adecuado de errores

Los tres casos de falla (conexión, formato, dato faltante) y la validación de entrada están contemplados, con mensajes claros, y probados uno por uno.

Con todo verificado, guarda este avance con un commit final, y súbelo a tu repositorio remoto:

```
git add .
```

```
git commit -m "Incorporar manejo de excepciones y validación al módulo"
git push
```



Esa captura debería mostrar tu terminal (o tu cliente de Git) confirmando el commit, con el mensaje que acabas de escribir, y el "push" ya reflejado en tu repositorio remoto de GitHub.

📦 Por qué este módulo es la base de lo que viene

Vuelve, por un momento, a la lista de lo que pide el **Gate 1** al final del **Módulo 1**: repositorio, entorno reproducible, una petición a un modelo, procesamiento de la respuesta, manejo de errores, un **README**, y un historial de commits. Antes de esta unidad ya tenías resueltas las primeras dos piezas; con el módulo que acabas de terminar, resolviste, en código real, dos piezas más: el procesamiento de la respuesta y el manejo de errores.

De las siete piezas del **Gate 1**, la que todavía no existe es el **README** que documenta todo esto para quien llegue a tu repositorio sin haber visto el proceso. Esa pieza, junto con el ensamble final de todo lo que construiste, es el trabajo que te espera en las unidades que siguen.

📤 Cómo entregas esto

➡️ El entregable exacto de esta unidad es tu módulo Python **dentro del repositorio**: el archivo en la carpeta "src" que consume el servicio externo, procesa la respuesta JSON, y gestiona los errores previsibles, ya confirmado y subido a tu remoto.

➡️ **En la plataforma**, pega el enlace de tu repositorio de GitHub como texto en el campo de entrega de esta unidad, igual que hiciste al cerrar la Unidad U1.2.

> 🚫 No corresponde adjuntar el archivo por separado ni pegar código suelto: lo que se revisa es el repositorio completo.

🧭 Siguiente lección

Bloque 8 · Tokens y request-response

Con tu módulo completo, organizado, verificado y protegido contra los errores más probables, cierras la **Unidad U1.3**. Cada pieza que construiste en estas tres unidades (tu entorno, tu setup técnico, y ahora tu primer módulo real) es, literalmente, parte del proyecto que vas a seguir construyendo durante el resto del programa.

Skillnest Learning Analytics

Bloque 7 - Incorpora excepciones y validación, y commitea tu avance [Core]

Tarea Core

Entregando como: Sebastián Eduardo Ayala Vásquez
ayala.phys.ds@gmail.com

Tu entrega

Subir archivo | Enlace (Drive, etc.)

Arrastra tu archivo aquí o haz clic para seleccionar

PDF, Word, Excel, PowerPoint, imágenes, ZIP — máx. 50 MB

Enviar entrega

Última modificación: miércoles, 7 de octubre de 2026, 16:59